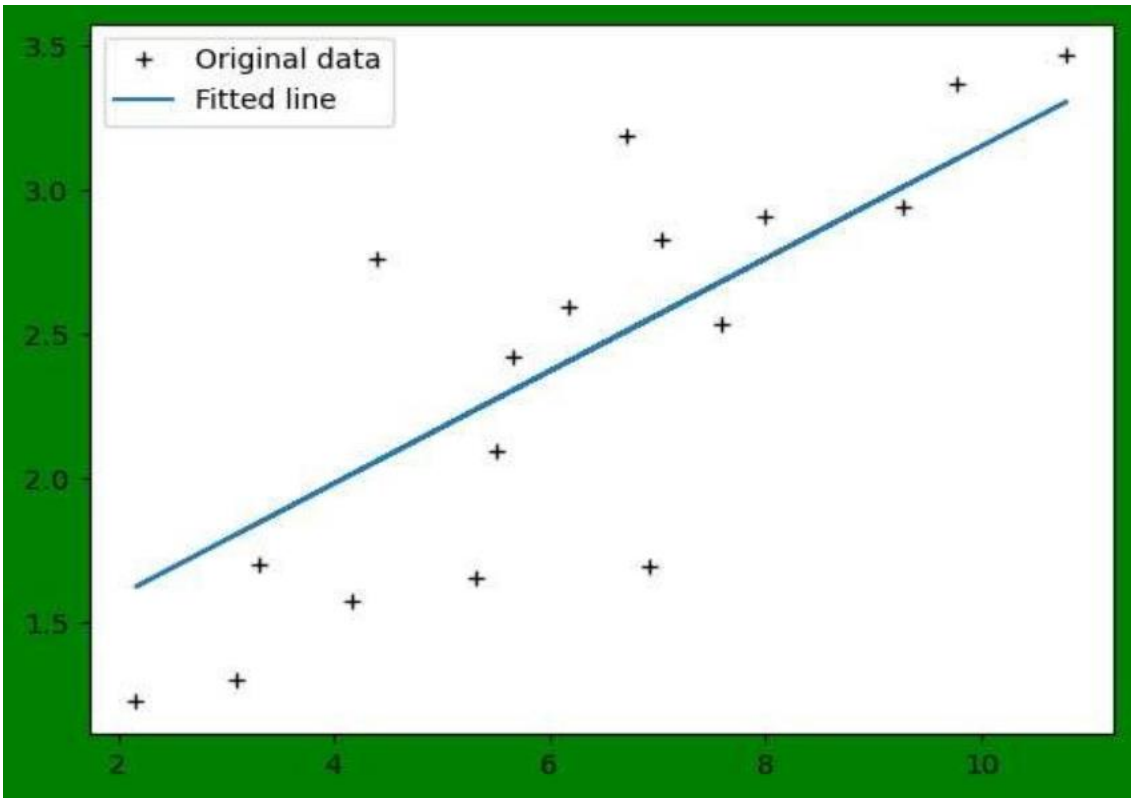
SID : 2534321

LAB1



LAB 2 :

```
Test Accuracy: 1.000
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 10)	350
dense_1 (Dense)	(None, 67)	737
dense_2 (Dense)	(None, 56)	3,808
dense_3 (Dense)	(None, 34)	1,938
dense_4 (Dense)	(None, 1)	35

Total params: 20,606 (80.50 KB)

Trainable params: 6,868 (26.83 KB)

Non-trainable params: 0 (0.00 B)

Optimizer params: 13,738 (53.67 KB)

LAB : 3

Test accuracy: 0.8736000061035156

```
[27]: print(model.summary())
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 56)	43,960
dense_1 (Dense)	(None, 45)	2,565
dense_2 (Dense)	(None, 32)	1,472
dense_3 (Dense)	(None, 24)	792
dense_4 (Dense)	(None, 10)	250

Total params: 49,039 (191.56 KB)

Trainable params: 49,039 (191.56 KB)

Non-trainable params: 0 (0.00 B)

None

LAB : 4

```
model = Sequential()

# Your code to add as many layers with activation functions of your choice:

model.add(Dense(200,activation='relu',input_shape=(n_features,)))
model.add(Dense(150,activation='relu',input_shape=(n_features,)))
model.add(Dense(100,activation='relu',input_shape=(n_features,)))
model.add(Dense(80,activation='relu',input_shape=(n_features,)))
model.add(Dense(40,activation='relu',input_shape=(n_features,)))


model.add(Dense(1))
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 200)	4,000
dense_1 (Dense)	(None, 150)	30,150
dense_2 (Dense)	(None, 100)	15,100
dense_3 (Dense)	(None, 80)	8,080
dense_4 (Dense)	(None, 40)	3,240
dense_5 (Dense)	(None, 1)	41

Total params: 60,611 (236.76 KB)

Trainable params: 60,611 (236.76 KB)

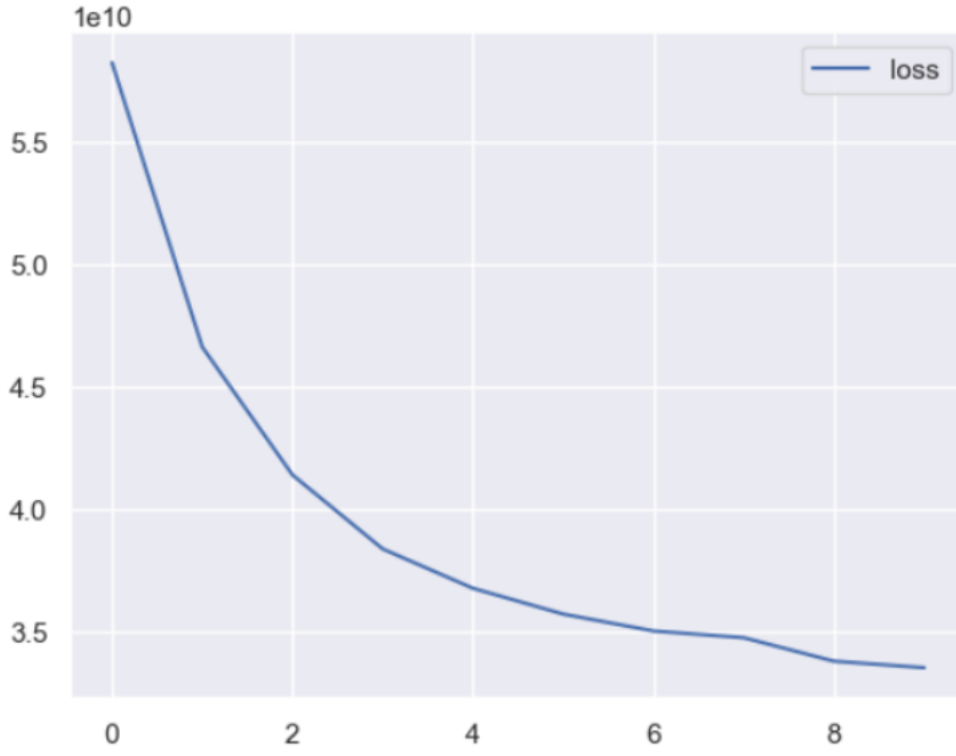
Non-trainable params: 0 (0.00 B)

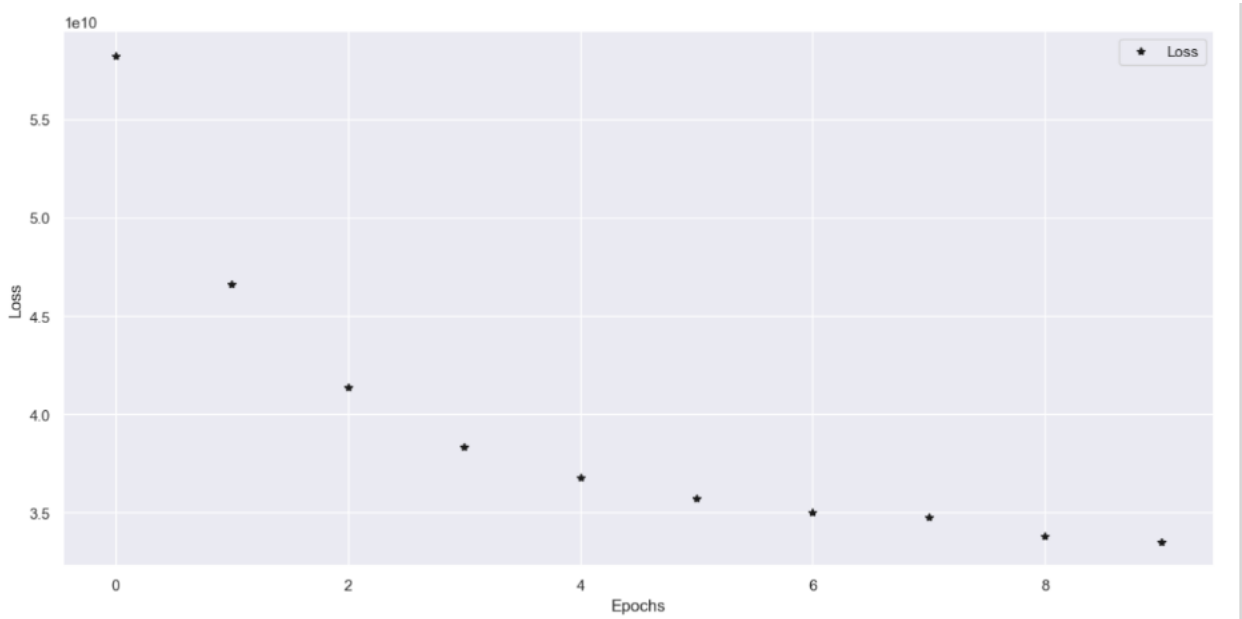
None

5]:
```
# ! add your chosen optimiser !

model.compile(optimizer='Nadam', loss='mse')
```

]: <Axes: >





LAB : 5

[55]:
```
model = Sequential()

# CONVOLUTIONAL LAYER
model.add(Conv2D(filters=32, kernel_size=(4,4),input_shape=(32, 32, 3), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

# Add more convolution and pooling layers with your choice of filters, strides, and activation function
# ...
# ...
model.add(Conv2D(filters=44, kernel_size=(4,4), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Conv2D(filters=56, kernel_size=(4,4), activation='relu',))
# POOLING LAYER
model.add(MaxPool2D(pool_size=(2, 2)))

model.add(Flatten())

# Add a few dense layers.
# ...
# ...
model.add(Dense(70, activation='relu'))
model.add(Dense(60, activation='relu'))
model.add(Dense(40, activation='relu'))

#LAST LAYER IS THE CLASSIFIER, THUS 10 POSSIBLE CLASSES
model.add(Dense(10, activation='softmax'))
```

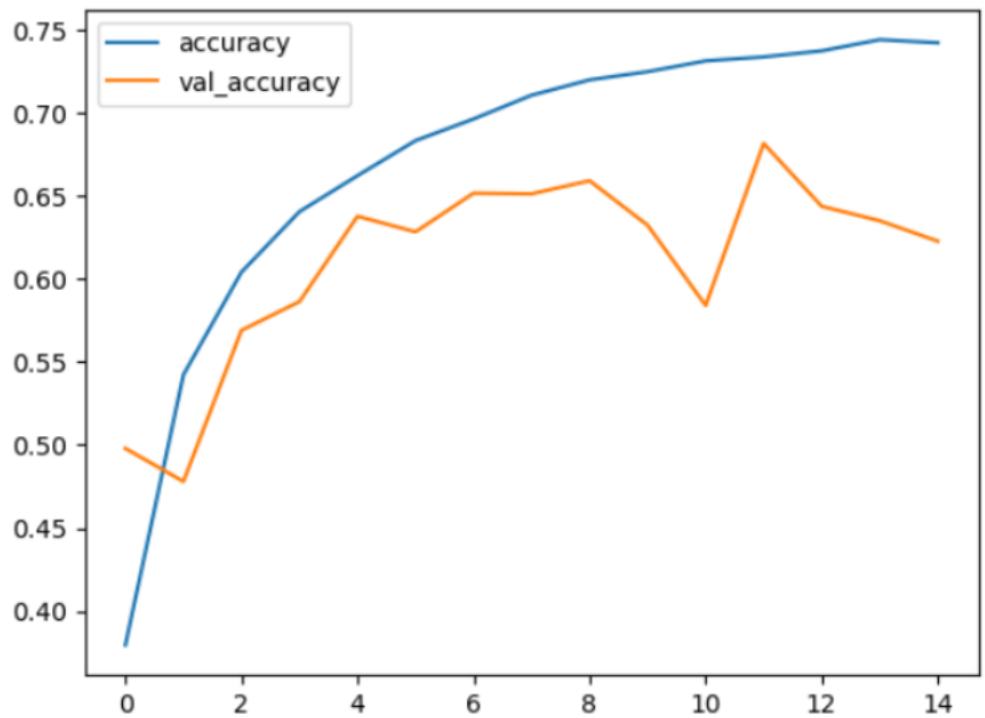
```
[56]: model.summary()
```

Model: "sequential_1"

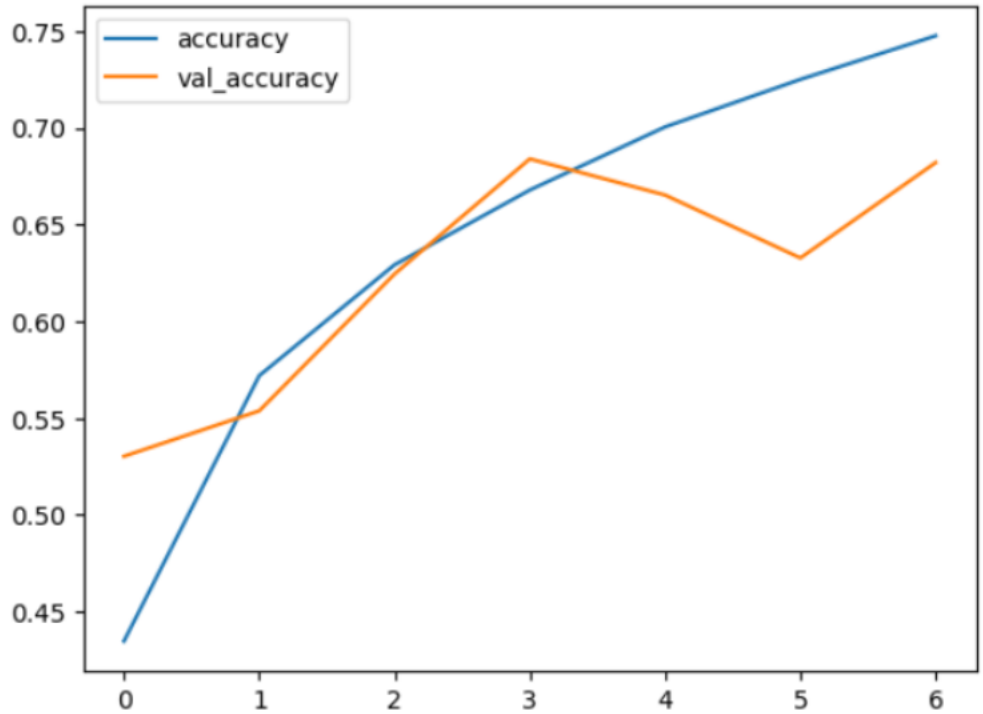
Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 29, 29, 32)	1,568
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_4 (Conv2D)	(None, 11, 11, 44)	22,572
max_pooling2d_4 (MaxPooling2D)	(None, 5, 5, 44)	0
conv2d_5 (Conv2D)	(None, 2, 2, 56)	39,480
max_pooling2d_5 (MaxPooling2D)	(None, 1, 1, 56)	0
flatten_1 (Flatten)	(None, 56)	0
dense_4 (Dense)	(None, 70)	3,990
dense_5 (Dense)	(None, 60)	4,260
dense_6 (Dense)	(None, 40)	2,440
dense_7 (Dense)	(None, 10)	410

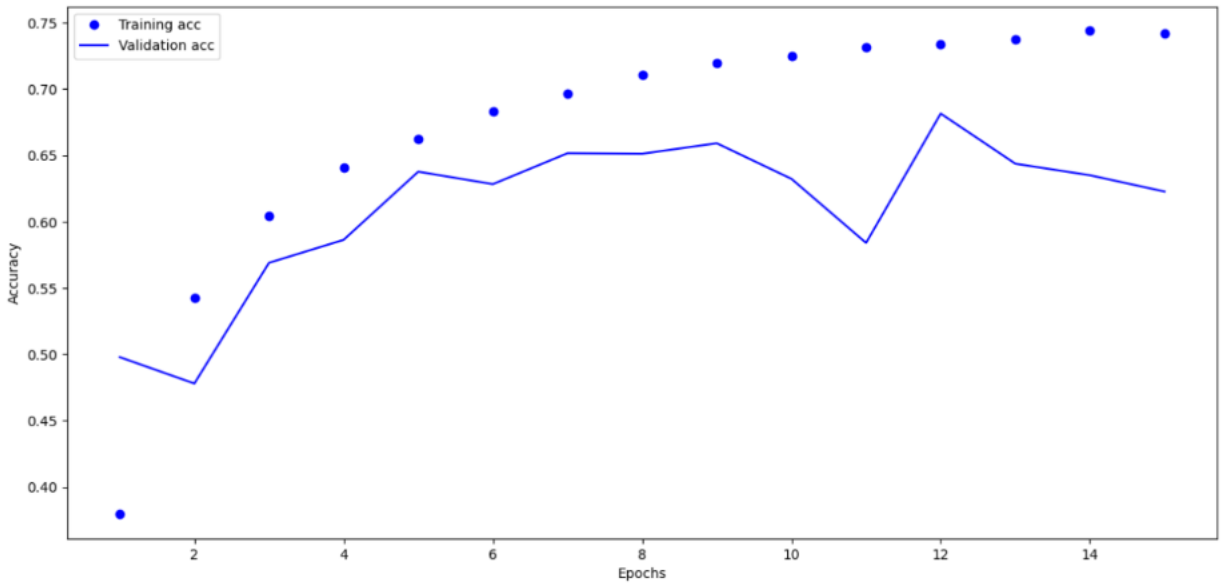
Total params: 74,720 (291.88 KB)

Trainable params: 74,720 (291.88 KB)

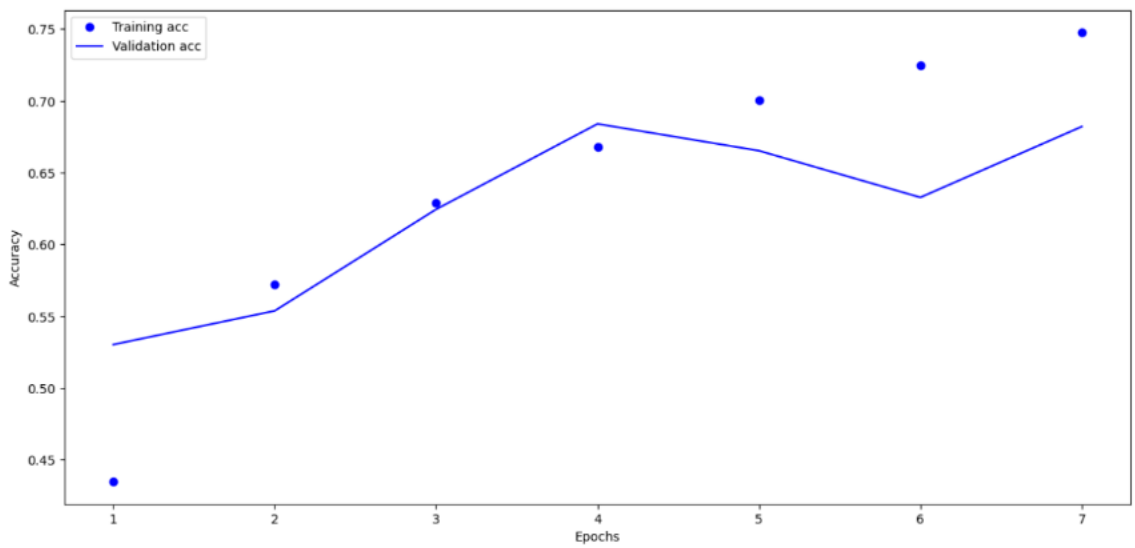


[91]: <Axes: >




Training acc
Validation acc
Accuracy
Epochs
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0.40
2
4
6
8
10
12
14

[96]:


Training acc
Validation acc
Accuracy
Epochs
0.75
0.70
0.65
0.60
0.55
0.50
0.45
1
2
3
4
5
6
7